読み上げ台本(日本語) — WireGuard for Apache NuttX (Community Over Code, Glasgow)

【2026-10-13 の注意】英語版は読み上げ用に作り直した(Essential / Optional / Presenter Notes の3区分、話速 125 語/分、短い文、アプリ版とデモの扱いの整理)。この日本語版はまだその書き直しに合わせていない。内容の対応が必要なときは英語版を正とする。

本番は英語で話す前提(デッキは英語、coc-glasgow-outline.md より "English, ~20-30 min")。この日本語版は英語版と 1対1で対応する**リハーサル・理解確認用**の台本 — 内容を体に入れる、または国内で 日本語版として話す場合のベース。英語版が更新されたら、この対応も崩れるので 両方直すこと。

本編35枚+付録2枚。ライブデモ150秒を含めて合計約29分(28:57)。2026-10-13、第7部を4枚に整理(Summary / What I learned / Acknowledgements / Thank You — Questions?): Giving back の独立スライドをやめて状況は Summary に(提出していないと明記)、Summary を発表の技術的な成果に絞り、謝辞は縦の一覧に、Alan の氏名を Alan Carvalho de Assis に直した。表紙と最終ページに同じスライドの QR コードを置き、古い動画のリンクを外した。2026-10-13、第6部(ライブデモ)を1枚に統合: 上に NETWORK SETUP(Laptop・Smartphone・Stack-chan と暗号化トンネル)、下に LIVE DEMO STEPS(4ステップ)、隅に Stack-chan の写真と GitHub リンク。Version の表記、Takeaway、バックアップ動画へのリンクは置かない。2026-10-13、第5部を4枚に整理: Version 1 / 2 の BEFORE / AFTER、ビルドモードの3行、図の USER SPACE / NUTTX KERNEL の2領域、「見つかった2つの問題」(暗号とスタックを別の問題として。スタックはこのドライバ自身)、検証は表にして GitHub の案内を削除。「ping が見逃すもの」と「時計を信じるか」は付録(A1・A2)へ移し、Thank You の後に置いた。2026-10-13、第4部(19〜23 枚目と目次)も整理: 目次の名称を Version 1: Application-Based Implementation に、目標と Takeaway の分離、層ごとの作業量(再利用・新規の行数)、バグ②③の色とラベル(SYMPTOM / ROOT CAUSE & FIX、QUICK FIX / REAL FIX)。2026-10-13、第2・3部(11〜13・16・17 枚目)も作り直し: 課題 → 手段 → 解決(機器に VPN)→ 開発の方針(wireguard-lwip の KEEP / ADAPT / REPLACE)→ lwIP と NuttX の対応、の流れに。2026-10-13、前半(2・5・6・7・9 枚目)を作り直し: 自己紹介は経歴と活動の写真を大きくし、概要は文章を短く、経緯は Version 1 → Version 2 の矢印つきの帯に、NuttX とは何かと比較(旧7・8枚目)は採用事例の写真つきの1枚に統合、経験は SPRESENSE の衛星プロジェクトと AITRIOS の2枚のカードにして、次の課題へつなぐ。2026-10-12、発表者の改善メモを受けて簡素化: 第4部の落とし穴は「6つのバグの一覧表」1枚 + 詳しく話す2件(ping は通るのに TCP が死ぬ / 50ms ポーリングで10倍遅い)にし、落とし穴1(detach された pthread)と4(実機のスタック)は 一覧表とノートへ。第5部は「なぜカーネルへ」の意図を明確にし、設計は図1枚に、検証は FLAT(実機)/ KERNEL(QEMU)/ PROTECTED(QEMU)を分けたカード3枚に、「ping が見逃すもの」と タイムスタンプは短く。第4・5部に GitHub への案内(QRコード付き)。第7部は運用性と移植性の 2枚を削り(要点はまとめのノートへ)、「What I learned」を1枚追加。2026-10-10、目次を7部(概要と背景 / 課題とアイデア / 計画 / 第1版 / 第2版 / ライブデモ / まとめ)に組み直し、第1版の動画と謝辞の2枚を追加。2026-10-10、ノートに小ネタ2つ(19・A2、斜体で出典を付記)。知らない人には普通の文として聞こえるものだけで、外しても流れは変わらない。時間は累積で、英語版と同じ値(英語の読み上げ速度で計算)。

重心はスライド28(検証) — 約1.8分。査読者が実際に確認してくる主張はここに集中している。**主張と注記(実機は FLAT / KERNEL と PROTECTED は QEMU のみ、実機の以前の版と 2026-10-12 の ESP32-S3 での提出シリーズそのものの実行の区別、PROTECTED の wg スタック不足とその修正、4 CPU は1回のスクリプト実行、KERNEL・PROTECTED は実機では未検証)は削らない。**直後にあった「ping が見逃すもの」と「時計を信じるか」は付録(A1・A2)に移した。本編の流れには入れず、時間にも含めない。質問で必要になったら、Thank You から下矢印で開く。

スライド2と4〜14(About Me、Overview〜Architecture。経緯と、NuttX の概要・採用事例の1枚を含む)も削らない。 NuttXとは何か、どこで 使われているか、Sonyでの歴史、自分がどこで出会ったか、そこからWireGuardに至った 理由 — この因果の連鎖が、後半すべての前提になる。**スライド33(What I learned)も残す** — 発表者自身のメッセージ。

時間が足りないときに削る順番:

1. スライド23(第1版の動画の静止画 — ライブデモが同じ種類のものを見せる)
2. スライド22(落とし穴 3/6、50ms ポーリング — 1行の教訓は一覧表にすでにある)

ライブデモが止まったら、すぐローカル録画へ切り替える。**目次に戻るスライド(4・10・15・18・24・29・31)は各10秒程度の一呼吸** — 読み上げる ものではなく、指し示して一拍置くだけ。本当に時間がなければ無言で次へ送ってもよい (時間短縮にはほぼ寄与しないので、切るのはここではなく上の3枚から)。

本番前に確認: 合計約29分は30分枠に収まる。本編は約28分を目標にリハーサルし、 上の削る順番を本番前に決めておく。採用事例のうち Fitbit の機種は Fitbit 自身の 2020 年の NuttX の講演、Xiaomi の 1,000+ は報告値、ARICA の写真は第三者の画像(許諾の確認が要る)。

---

1 — タイトル (0:00 → 0:49)

画面: 「WireGuard for Apache NuttX」を大きく。その下にサブタイトル「Bringing Secure Remote Access to Embedded Devices」。上に Community Over Code Glasgow 2026、下に Satoru Akita / Sony Semiconductor Solutions。右下に小さな QR コード (「Slides」、最後のスライドと同じ URL)。技術的な情報(wg0、QEMU、ビルドモード、検証環境、バージョン)は載せない。

話す:

- 皆さん、こんにちは。日本の Sony Semiconductor Solutions から来ました、Satoru Akita です。
- 今日は、軽量な VPN の WireGuard を Apache NuttX に持ち込んだ経験を話します。
- 目的はシンプルでした: 組み込み機器への、安全な遠隔アクセス。
- なぜこのプロジェクトを始めたか、どう作ったか、直面した難しさ、そして学んだことを話します。

2 — 自己紹介 (0:49 → 1:48)

画面: 見出し「About Me」。上: 本人写真・名前の1行・SNS の狐アイコン。下の左: 日本地図 (① 名古屋 生まれ → ② 仙台 大学 → ③ 川崎 現在)と、その下に「Japan」のラベル。下の右: 「My background & activities」の大きな写真5枚 (Rocket / Aerospace Engineering: モデルロケット / DNA Nanorobotics: cryo-TEM 像 / Semiconductor Research: ウェハ上の試作チップ / Robots: ハッカソンのヒューマノイド / Open-Source Conference Volunteer: 運営参加時の写真)。活動を説明する文章はない。経歴の流れ(分子ロボティクス → … → NuttX)は口頭だけ。

話す:

- 簡単に自己紹介します。
- Sony Semiconductor Solutions で edge AI エンジニアをしていて、application 側から Apache NuttX に関わっています。日本から来ました: 名古屋生まれ、大学は仙台、今は川崎に住んでいます。
- 経歴は幅広いです。大学では機械・航空宇宙工学を学び、ロケットにも取り組みました。学部の研究は DNA ナノロボティクス、修士の研究は半導体でした。ハッカソンではロボットも 作り、Open Source Summit Japan や

KubeCon など、オープンソースのカンファレンスにも 運営ボランティアとして参加しています。SNS では、この写真の狐が私で、オープンソースの本も 書きました。

- ここに至る道のりは、分子ロボット工学の世界大会、コンピュータビジョンと AI システムの 仕事、edge AI カメラ、そして SPRESENSE 上の embedded systems を通っています — NuttX と WireGuard は、その最新の到達点であって、出発点ではありません。
- ここではっきりさせておきたいことがあります: 私は NuttX の kernel developer として 始めて、VPN を追加しようと思い立ったわけではありません。私は application 側・deployment 側から NuttX に繰り返し出会い続けていて、ある時、その立場からは 解決できない課題に行き当たりました。それが今日の話です。 次は、プロジェクトの概要です。

3 — 目次 (1:48 → 2:27)

画面: 7つの部。見出しは短い名詞句、その下に「これから何を話すか」を予告する短い1行、右に所要時間 — Overview & background: Project origins · NuttX & Sony · my experience(~5分)/ The Problem & Solution: Remote access challenge · why WireGuard(~4分)/ The Development Plan: Reuse wireguard-lwip · adapt to NuttX(~2分)/ Version 1: Application-Based Implementation: First working tunnel · debugging challenges(~4分)/ Version 2: The Kernel Driver: Why a driver · integration challenges · validation(~5分)/ Live Demo: Secure remote access · Stack-chan control(~3分)/ Summary & Reflections: Achievements · personal reflections · community(~4分)。

話す:

- 今日の話の地図です。
- 7つの部があります。まず背景: このプロジェクトの始まり、NuttX と Sony、そして私自身の経験。次に、課題 — 組み込み機器への安全な遠隔アクセス — と、WireGuard を選んだ理由。3つ目が開発計画: wireguard-lwip を再利用して、 NuttX に合わせる。4つ目は Version 1、アプリケーションとしての実装: 最初に動いたトンネルと、デバッグの難しさ。 5つ目は Version 2、カーネルドライバ: なぜドライバか、統合の難しさ、そして検証。6つ目は、安全な遠隔アクセスのライブデモで、 Stack-chan を操作します。最後に、まとめ: 成果、私の振り返り、そしてコミュニティ。
- 全部で約30分、その後質疑応答です。
- この後、部が変わるたびにこの目次に戻ってきて、今どこにいるかを示します。

4 — 目次 ①Overview & background (2:27 → 2:37)

画面: 目次と同じリスト。①Overview & background だけハイライト、他は薄い。

話す(一言): では、第1部の概要と背景から。

間: ハイライトされた行を指して一拍。

5 — プロジェクト概要 (2:37 → 3:08)

画面: `wg0` netdev の図(変更なし)— アプリケーション/NSH、NuttX ネットワークスタック、 `eth0`/`wlan0` の隣に `wg0`、その下に UDP ソケット、インターネット、ピア。図の下にカード3枚: Lightweight VPN / Virtual interface / Secure communication。最下段に一文。

話す:

- 全体像を1枚の絵にするとこうなります。
- 軽量な VPN の WireGuard を Apache NuttX に持ち込みました。`wg0` という仮想ネットワーク インターフェースとして、普通のインターフェースの隣に現れます。`wg0` に送ったパケットは 暗号化されて UDP ソケットで運ばれ、反対側は普通の WireGuard ピアなら何でも構いません。これで、NuttX のマイコンと安全に IP 通信できます。
- 技術的な詳細は後半に回します。まず、このプロジェクトがどう始まったかです。

6 — 経緯: 2月の思いつきから10月の本番まで (3:08 → 4:06)

画面: 2月〜10月の月の横軸。上に出来事 — 2月 中旬 GSoC のテーマ一覧を見て調べ始める / 3月 上旬 計画を提案 / 3月20日 Alan さんからこのカンファレンスを聞き、CfP を提出 / 4月 上旬 GSoC に応募 / 5月 上旬 CfP は採択、GSoC は不採択 — それでも続ける / 10月 グラスゴー。下に、矢印の形をした3本の帯 — 5〜7月 NuttX と WireGuard を調べ、作り方を探る → 8月 Version 1(アプリ・FLAT・実機検証)→ 9月〜 Version 2(カーネルドライバ)。

話す:

- このプロジェクトがどう進んだかを、月ごとに見せます。
- 2月の中旬に GSoC のテーマ一覧を見て、調べ始めました。3月に計画を提案し、3月20日に Alan さんからこのカンファレンスのことを聞いて、CfP(発表の募集への応募)を出しました。4月の初めに GSoC に応募しました。この2つは別々の応募です。
- 5月の初めに2つの返事が来ました。CfP は採択、GSoC は不採択。それでも続けることに しました。5〜7月は NuttX の内部を調べ、WireGuard の仕組みを学び、作り方を探りました。8月は実機での検証を中心に、Version 1 — FLAT ビルドのアプリ — を作って確かめました。その後に Version 2、カーネルドライバが続きます。
- GSoC をきっかけに始めた開発が、一歩ずつ育ち、今も続いています。次は NuttX とは何か、です。

7 — A Quick Overview of Apache NuttX: NuttX とは何か、どこで使われているか (4:06 → 5:08)

画面: 左: NuttX のロゴと4つの要点(POSIX-like な RTOS / 小さい / 幅広いハードウェア対応 — 15+ アーキテクチャ・300+ ボード / Apache のオープンソース)、標準 API の小さな表 (FreeRTOS: 独自のタスク API / Zephyr: 独自 API とオプションの POSIX 層 / NuttX: POSIX-like が設計方針)。右: 実機の写真タイル — Sony、Fitbit(Charge 3 / 4)、PX4 のフライトコントローラ、月面ロボット SORA-Q、Xiaomi OpenVela(1,000+ 製品)。最下段:「Apache NuttX is a POSIX-like RTOS used in a wide range of real-world embedded devices.」

話す:

- では、Apache NuttX とは何でしょうか。
- NuttX は、小さなデバイス向けの POSIX-like なリアルタイム OS です。フットプリントが小さく、ハードウェア対応がとても広い — 15 以上の CPU アーキテクチャと 300 以上のボードです。Apache の top-level project でもあり、公開の場で開発されています。
- FreeRTOS や Zephyr も良い RTOS です。私が重視する違いは API です。NuttX は設計として POSIX-like で、pthread、ソケット、ファイル、シェルがあります。だからここでは、VPN を 普通のネットワークデバイスにできます。
- そして、おもちゃではありません。Sony は音声製品と SPRESENSE で使っています。Fitbit は Charge 3 や Charge 4 などのフィットネストラッカーで使いました。PX4 のドローン用 フライトコントローラも NuttX で動きます。Xiaomi の OpenVela は 1,000 を超える製品で 動いていると報告されています。そして小さな月面ロボット SORA-Q が、日本の 2024 年の 月面ミッションで使いました。次は、Sony 自身の NuttX の歴史です。

8 — History: Sony と NuttX の歴史 (5:08 → 6:39)

画面: 縦タイムライン — 2015 Sony audio products / 2018–19 SPRESENSE・CXD56xx (OSS+ELC Europe 2019 で発表) / 2020 upstream collaboration(NuttX Online Workshop、「A Journey from Fork to Mainline」) / 2020s sensing & edge devices。

話す:

- これは Sony の NuttX との歴史であって、まだ私の話ではありません — そして edge AI より長いです。
- Sony は 2015 年ごろから、NuttX を使った音声製品を出荷しています。数年後に登場した SPRESENSE とその CXD56xx チップは、音声、GNSS、カメラ、マルチコアのサポートを NuttX に持ち込み、2019 年に Sony の Masayuki Ishikawa 氏が Open Source Summit + Embedded Linux Conference Europe でその成果を発表しています。そこから、Alin Jerpelea 氏を含む Sony のエンジニアたちが、upstream への大きな貢献者に なっていきます — 後に NuttX の project management committee の議長も務める 人物です。彼の 2020 年の講演は文字通り「A Journey from Fork to Mainline (fork から mainline への旅)」という題でした。
- 一人のエンジニアがいたから Sony が NuttX を採用した、という因果関係は主張しません — それを裏付ける一次情報がないからです。公開されている講演や文書から言えることは もっとシンプルです: Sony の NuttX 利用は edge AI より何年も前からあり、カメラや GNSS、センシングの仕事へと時間をかけて広がっていった。これが、私自身の経験が 立っている土台です。

9 — Experience: SPRESENSE と AITRIOS での私の経験 (6:39 → 7:40)

画面: 見出し「My Experience with NuttX」。写真つきのカード2枚。SPRESENSE — Satellite Project(人工衛星 ARICA の写真)/ AITRIOS — Edge AI Sensing Platform (edge AI カメラの写真)。最下段:「I wanted a secure way to access and troubleshoot remote embedded devices.」

話す:

- 今のが Sony の歴史です。次は、私自身の NuttX の経験です。
- まず SPRESENSE。Sony の低消費電力のマイコンボードで、NuttX が動きます。青山学院大学の 人工衛星 ARICA-2 は、ガンマ線バーストの速報を送る小さな衛星で、SPRESENSE を搭載 コンピュータに使っています。2026 年 4 月に打ち上げられました。私は社内のボトムアップの 技術活動として、このプロジェクトに参加しました。
- 次に AITRIOS。Sony の edge AI センシングプラットフォームで、edge AI カメラの管理や エッジでの推論に使われます。私は edge AI カメラのシステムの導入と運用に関わってきました。私が関わったシステムでは、ボードは ESP32 で、Apache NuttX が動いていました。AITRIOS のカメラがすべてこの構成というわけではありません。
- 軌道上の衛星も、現場に置かれたカメラも、私の机からは遠い場所にあります。この経験から、遠くの組み込み機器に安全にアクセスして、困ったときに調べられる手段がほしいと思うように なりました。それが、次に話す課題です。

10 — 目次 ②The Problem & Solution (7:40 → 7:50)

画面: 目次と同じリスト。②The Problem & Solution だけハイライト、他は薄い。

話す(一言): 背景は以上です。第2部: 課題と、私が選んだ解決方法。

間: ハイライトされた行を指して一拍。

11 — Problem: 遠隔の機器へ、どう安全にアクセスするか (7:50 → 8:43)

画面: 見出し「Problem: How Can We Securely Access Remote Devices?」。Engineer → Internet → Router/network → Wi-Fi → NuttX device の図(変更なし)。経路全体に必要な安全な道と、WPA2/WPA3 が守る範囲(Wi-Fi の一区間のみ)の対比。下に3枚のカード: Devices are far away / Remote diagnosis / A secure path。最下段:「We needed a secure way to access and troubleshoot remote NuttX devices.」

話す:

- ここから、私が解きたかった課題です。
- 組み込み機器は、遠くに設置されることがよくあります — 顧客の現場、工場、別のネットワーク。何か起きたとき、技術者は現地へ行かずに、ログインして機器を調べ、直したいのです。
- Wi-Fi のセキュリティ、WPA2 や WPA3 は、機器とアクセスポイントの間の無線リンクを 守ります。大事ですが、守るのは一区間だけです。インターネットを越えて機器に届く 方法そのものを与えてくれるわけではありません。そのためには、技術者から機器までの 全体をつなぐ、安全な経路が別に要ります。同じ必要は、edge AI や産業用 IoT など、遠隔のシステム全般にあります。
- そこで問いは、「遠隔の機器にどう安全にアクセスするか」です。方法を見てみましょう。

12 — Options: 遠隔の機器へ届く方法 (8:43 → 9:35)

画面: 見出し「Options: Ways to Reach a Remote Device」。6つの手段(ポートを開ける / クラウド経由の HTTPS・MQTT / リバーストンネル・リレー / ルーターの VPN / キャリアの閉域網 / 機器の VPN)を、任意の IP サービスに届くか・間にサーバーが要らないか・どこまで守られるか、で比べた表。最後の行(機器の VPN)が今回の選択。最下段:「Every option has trade-offs. I chose a VPN on the device, so the device itself is the end of the tunnel.」

話す:

- 遠隔の機器へ届く方法はいくつかあります: ポートを開ける、クラウド API や MQTT ブローカーを使う、リバーストンネルを張る、ルーターの VPN を使う。
- どれにも得失があります。HTTPS と MQTT はデータを送るには優れていますが、機器上のシェルや Web ページは得られません。ルーターの VPN はルーターが自分のものなら良い方法ですが、守りは ルーターで終わります。どれも悪いわけではなく、どれも安全にできます。守る範囲が違うだけです。
- このプロジェクトでは別のものがほしかった: NuttX の機器の上で直接動く標準の VPN。機器自身がトンネルの終点になります。それが解決方法です。

Tailscale や ZeroTier について聞かれたら: これらも機器上の VPN だが、今のところ マイコンより大きな OS が要ることが多い(スライドの脚注)。

13 — Solution: 機器に VPN を載せる (9:35 → 10:35)

画面: 見出し「Solution: A VPN on the Device」。Engineer/Linux/Windows ↔ WireGuard トンネル ↔ NuttX device の図(変更なし)。その下に小見出し「Why WireGuard?」と3枚のカード: Small & simple(コード行数の小さな棒グラフ: IPsec 約42万 / OpenVPN 約12万 / WireGuard 約3,800、2018年)/ Modern cryptography(ChaCha20-Poly1305、Curve25519)/ UDP-based tunnel。最下段:「Run a lightweight, standard VPN directly on the NuttX device.」

話す:

- では私の解決方法です: 機器自身に VPN を載せる。
- なぜ WireGuard か。まず、小さくてシンプルです。2018 年の講演で、作者がコード行数を比べました: WireGuard は約 4,000 行、OpenVPN は約 12 万行、strongSwan を使う IPsec は 40 万行超。特定の実装の古い数字ですが、差は大きく、マイクロコントローラでは意味があります。
- 次に、モダンな暗号: ChaCha20-Poly1305 と Curve25519。詳細はここでは話しません。そして、素の UDP の上のトンネルです。ピアは公開鍵で識別され、パケットは普通の UDP なので、Linux や Windows など既存の WireGuard ピアと話せます。マイクロコントローラ自身が、トンネルの終点になります。
- 考え方は単純です: 軽量で標準の VPN を、NuttX の機器の上で直接動かす。次は、`wg0` が NuttX のどこに入るかです。

本番前に確認: 行数は WireGuard 作者の Linux Plumbers 2018 の講演スライドから。古く、特定の実装の 数字で、同じ講演の別のスライドは OpenVPN の数字が少し違う。聞かれたらそう答える。

14 — Architecture: NuttX ネットワークスタック上の WireGuard (10:35 → 11:23)

画面: Application → TCP/UDP/IP → wg0 (NET_LL_TUN netdev) → WireGuard encrypt/decrypt → UDP socket → wlan0/usrsock・物理ネットワーク、の層状図。左右に TX/RX 両方向の矢印。

話す:

- 概要図より一段技術的になります。ここから実装の話が始まります。
- 送信方向では、アプリケーションから TCP または UDP・IP を通って降りてきた パケットが、他の netdev と同じように `wg0` に届き、WireGuard によって 暗号化され、ボード上では `wlan0` を通る、あるいは Wi-Fi そのものがソケット ベースの周辺機器である `usrsock` を通る UDP データグラムとして出ていきます。受信方向はこの逆です: UDP データグラムが届き、WireGuard が復号し、普通の インターフェースに届いたかのように `wg0` へ渡されます。
- ここから先はすべて、この真ん中の部分 — `wg0` と暗号化のステップ — を、上下の層に触れずに作る話です。

15 — 目次 ③The Development Plan (11:23 → 11:33)

画面: 目次と同じリスト。③The Development Plan だけハイライト、他は薄い。

話す(一言): 第3部: 開発の方針。既存の WireGuard から、どう始めるか。

間: ハイライトされた行を指して一拍。

16 — Porting Strategy: wireguard-lwip から始める (11:33 → 12:38)

画面: 見出し「Porting Strategy: Start from wireguard-lwip」。一番上に GitHub のリンク (github.com/smartalock/wireguard-lwip、クリックできる)と「An existing open-source WireGuard for lwIP」。その下に大きな3段: KEEP — Protocol & crypto(緑、小さな修正2件つき)/ ADAPT — Platform hooks(金)/ REPLACE — lwIP network glue(赤)。最下段に一文。

話す:

- ここから開発の方針です。WireGuard をゼロから書いたわけではありません。

- wireguard-lwip から始めました。lwIP ネットワークスタック向けの、オープンソースの WireGuard 実装です。リンクはスライドにあります。プロトコルも暗号もすでにあるので、移植の作業に集中できました。
- コードを3つに分けました。1つ目は、プロトコルと暗号: ハンドシェイク、データ転送、暗号の部品。これは残しました。暗号はそのまま使い、プロトコルの核には、後でレビューをして見つけた小さな修正が 2件要りました。2つ目は、プラットフォームフック。OS に依存するものは1つの小さなヘッダの裏に 4つの関数としてまとまっています: 単調な時計、乱数、TAI64N 形式の現在時刻、システムが高負荷かの 確認。これらを NuttX 向けに合わせました。3つ目は、lwIP の接着部分。NuttX には lwIP がないので、この層は置き換えました。ここが本当の移植作業です。
- 次は、lwIP の考え方を NuttX にどう対応づけたかです。

17 — Mapping lwIP Networking to NuttX (12:38 → 13:24)

画面: 見出し「Mapping lwIP Networking to NuttX」。lwIP と NuttX の対応表 — ネットワークデバイス(`struct netif` → `struct net_driver_s`)/ パケットバッファ(`pbuf` → `iob`)/ 登録(`netif_add()` → `netdev_register()`)/ 送信(output コールバック → `d_txavail` → `devif_poll()`)/ UDP の「ワイヤ」(`udp_pcb` → UDP ソケット)。注記「考え方の対応で、API の1対1の置換ではない」。最下段:「Porting meant using NuttX's own building blocks, not bringing lwIP into NuttX.」

話す:

- これが移植で一番大事な判断でした。
- lwIP を NuttX に持ち込む代わりに、lwIP の考え方を1つずつ NuttX の部品に対応づけました。lwIP のネットワークインターフェースは NuttX の `net_driver_s` になります。lwIP のパケットバッファ `pbuf` は、NuttX の I/O バッファ `iob` に当たります。インターフェースの追加はデバイスの登録になり、 `wg0` はトンネル型のデバイスとして登録します。lwIP が出力コールバックでパケットを送るところは、NuttX ではドライバの `txavail` 関数が呼ばれ、ドライバが `devif_poll` でスタックに問い合わせます。 `wg0` の下のワイヤは、素の UDP ソケットです。
- これは考え方の対応で、API の1対1の置き換えではありません。要点は単純です: NuttX 自身の部品を 使う。次は、Version 1、最初に動いたトンネルです。

18 — 目次 ④Version 1: Application-Based Implementation (13:24 → 13:34)

画面: 目次と同じリスト。④Version 1: Application-Based Implementation だけハイライト、他は薄い。

話す(一言): 第4部: Version 1、アプリケーションとしての実装。動いた、そしてバグが出てきました。

間: ハイライトされた行を指して一拍。

19 — Version 1: NuttX のアプリケーションとしての WireGuard (13:34 → 14:37)

画面: 見出し「Version 1: WireGuard as a NuttX application」。「GOAL」の枠(Run a standard WireGuard tunnel as a NuttX application. / Reuse the existing core and implement the NuttX integration.)。その下に3列の表(Layer / Work needed / In this port): Protocol & crypto — Almost none、暗号はバイト単位で同一、プロトコル本体は 小さな修正2つ(約3,100行を再利用・約40行を変更)/ OS hooks — Four functions(約120行を新規)/ Network glue — The real work、netdev + UDP socket + RX task(約1,200行を新規)。最下段は2行の Takeaway: Version 1 proved that WireGuard could run on NuttX. / It also exposed OS and networking problems that the first tunnel did not reveal.

話す:

- Version 1 は、WireGuard を NuttX のアプリケーションとして、FLAT ビルドに入れたものです。目標は、ファームウェアを作り直さずに、実行時に設定できる標準のトンネルでした。
- この表は、各層にどれだけの作業が要ったかを示します。wireguard-lwip からプロトコルと暗号処理が約3000行 来ました。暗号はそのままです。プロトコル本体はレビューの後で小さな修正が2つ必要で、そのうち1つは wireguard-lwip 自身のバグでした — レート制限が一度も効いていなかったのです。4つの OS 関数は小さく、約120行です。本当の作業はネットワークのつなぎ込み — netdev、UDP ソケット、受信タスク — で、最初に動いた トンネルでは約1200行の新規コードでした。
- 結果は、動くトンネルでした。それでやり方が正しいことは分かりましたが、最初の ping 成功では見えなかった OS とネットワークの問題も出てきました。どこが難しかったかを見ていきます。
- ここまでは成功譚に見えます — でも、お姫様は別の城にいました *(マリオの「Your princess is in another castle」)*。

本番前に確認: 行数は物理行(コメント込み)で、git の履歴から数えた。つなぎ込みは最初に動いたトンネル (2026-08-13)の値で、今のファイルは実行時設定などで約2倍の長さになっている。

20 — Where it was hard: 6つのバグ (14:37 → 15:46)

画面: 見出し「Where it was hard: six bugs」。6つのバグの一覧表 (バグ / 出た場所 / 教訓): 1 `wg` コマンドが終わると受信スレッドが止まる / 2 ping は通るのに TCP が死ぬ (in detail) / 3 「実害なし」が10倍(in detail) / 4 スタック94%、設定保存でボードが固まる、実機だけ / 5 NuttX 自身の ChaCha20-Poly1305(part 5) / 6 3KB のカーネルスタックに 6KB の配列(part 5)。GitHub への案内と QR コードは置かない。Takeaway: 多くは通る試験の陰に隠れていた。

話す:

- ここからは、苦労したところです。
- 最初のトンネルはすぐに動きました。時間の大半を使ったのは6つのバグです。 apps 版で4つ、カーネルに移してからさらに2つ。多くは、ハンドシェイクや ping のような通る試験の陰に隠れていたので、最初は違う場所を探していました。
- バグ1: 受信スレッドは detach 付きで作られていて、それを起動した `wg` コマンドが 終わると、一緒に止まりました。detach が決めるのはメモリがいつ解放されるかだけで、スレッドがどれだけ生きるかではありません。カーネル版でスレッドをドライバが 持っているのは、これが理由です。バグ4: 実機だけが見せてくれました。ボード上で、受信タスクのスタックは94%使われていて、残りは176バイトだけでした。sim はスタックが 大きいので、一度も見えませんでした。これは覚えておいてください。
- このうち2つを詳しく話します。その教訓は誰にとっても役に立つからです。バグ5と6は 第5部で出てきます。6つすべてのログと修正は GitHub のリポジトリにあります。

バグ1と4には、もう専用のスライドがない。 話は上のノートにある。詳しい版(バグ1の `SO_RCVTIMEO` の回り道を含む)は Q&A 資料にある。

21 — バグ②: ping は通るのに TCP が死ぬ (15:46 → 16:49) *(お気に入り)*

画面: 見出し「Bug ②: Ping Works. TCP Dies.」(オレンジ〜赤)。左の枠 SYMPTOM: ping は 0% loss / TCP のハンドシェイクは完了し、その後で接続が死ぬ / `EBADF` — “Bad file descriptor”。右の枠 ROOT CAUSE & FIX: `wg_rx` タスクからの送信は成功、TCP のデータを送るシステムのワークキューからは失敗 / 原因: ファイルディスクリプタは1つのタスクグループに属し、別のタスクグループでは有効でない / 修正: `struct socket` に対する `psock_*()` を使う(fd なし、どのタスクからも使える)。Takeaway は「If only some tasks fail, look at what each task owns. Here, it was the file descriptor.」

話す:

- このプロジェクトで一番好きなバグです。
- 同じトンネルの上で、ping は完璧に通り、TCP のハンドシェイクすら完了する — それなのに接続は 死にました。送信経路にログを足すと `EBADF`、不正なファイルディスクリプタが出ていました。成功していた 送信はすべて受信タスク自身の中で行われたものでした — ハンドシェイクの応答、キープアライブ、ping の応答、SYN-ACK(TCP 接続の2番目のステップ)。失敗していたのは、TCP のアプリケーションデータを後から送る システムのワークキューからの送信でした。
- 原因: NuttX では、ファイルディスクリプタはそれを開いたタスクグループに属します。fd が壊れていた のではなく、別のタスクグループからは見えなかったのです。修正は fd をやめ、NuttX の内部ソケット API を 使うことでした — これは `struct socket` に対して動作する、ただのメモリなので、どのタスクからでも使えます。カーネル版がソケットを `struct socket` として保持しているのは、まさにこの理由です。

22 — バグ③:「実害なし」が実は10倍遅かった (16:49 → 17:31) *(削る候補3)*

画面: 見出し「Bug ③: "No Real Impact" Was 10× Slower」(オレンジ〜赤)。2つの枠を並べる: QUICK FIX — Check every 50 ms(測定: 約10倍遅い)/ REAL FIX — Event-driven wake-up (`psock_poll()` のコールバックとセマフォ)。Takeaway: 測らない応急処置はタダに見える。測ること。

話す:

- 3つ目: 測定しなかった前提。
- fd を使わない修正は `poll` も壊しました。`poll` は fd ベースだからです。応急処置として 50ミリ秒ごとに ソケットを確認しました。コストはアイドル時に CPU を起こすことだけだと思っていました。間違いでした。 届いたパケットはすべて最大 50ミリ秒待たされ、それだけで速度の上限になっていました — 本来より約10倍遅い。 実害なしだと思っていたものが、実は10倍でした。
- 本当の修正は、正しい待ち方です。ソケットがコールバックし、データが届いたらセマフォがスレッドを起こします。

23 — Version 1 のデモ: 最初の動画 (17:31 → 17:57) *(削る候補2)*

画面: 左に動画の1コマ(公式 Windows WireGuard クライアントと、ESP32-S3 上の NuttX への telnet セッションが並んだ画面)。右に手順 — telnet 10.10.0.2 / uname -a、ifconfig / webserver & / ブラウザで http://10.10.0.2/ — と条件(ESP32-S3 DevKit、apps 版 (FLAT)、USB は抜いてある、相手は 公式 Windows クライアント)、小さく youtu.be/1kyX2av5WG4。Takeaway: Version 1 was enough for a demo. Next, I made it a real NuttX driver.

話す:

- Version 1 の動画の1コマです: トンネル越しに telnet でログインし、コマンドをいくつか打ち、ボード上の 小さな Web ページを開きました。ボードは電源アダプタだけで動いていて、相手は公式の Windows 版 WireGuard です。 Version 1 でもデモには十分でした。次に、これを本物の NuttX のドライバにしました。

再生しない: 静止画で十分。動画はライブデモの予備として取っておく。**これは apps 版。** この後のカーネル版の結果と混ぜて話さない。

24 — 目次 ⑤Version 2: The Kernel Driver (17:57 → 18:07)

画面: 目次と同じリスト。⑤Version 2: The Kernel Driver だけハイライト、他は薄い。

話す(一言): 第1版は以上です。第5部、難しい後半: 本物のカーネルドライバと、その確かめ方。

間: ハイライトされた行を指して一拍。

25 — Version 2: NuttX のカーネルドライバとしての WireGuard (18:07 → 19:13)

画面: 見出し「Version 2: WireGuard as a NuttX kernel driver」。左右2枚の枠 — 左 BEFORE — VERSION 1: 「An application」(`apps/` にあり、カーネル内部(`psock_*`)に手を入れる。FLAT ビルドでしか 動かず、in-tree のドライバとしては合わない)/ 右 AFTER — VERSION 2: 「A kernel driver」(`drivers/net/wireguard/` に `wg0`、小さな `wg` コマンドが ioctl で設定。FLAT・PROTECTED・KERNEL のために作った)。その下に BUILD MODES の3行: FLAT — アプリとカーネルが1つのアドレス空間 / PROTECTED — MPU がアプリとカーネルの メモリを分ける / KERNEL — MMU が別々のアドレス空間を与える。最下段: 「The goal: a proper kernel driver for every NuttX build mode, in a form that is easier to review upstream.」

話す:

- なぜカーネルに移したのか。
- Version 1 はアプリケーションです。`apps` にあり、カーネルのソケットの内部に手を入れています。アプリと カーネルが1つのアドレス空間を共有する FLAT ビルドなら、それで動きます。でも in-tree のドライバとしては 合わず、アプリとカーネルが分けられる PROTECTED や KERNEL ビルドでは動きません。PROTECTED では MPU が 両者のメモリを分け、KERNEL では MMU がそれぞれにアドレス空間を与えます。
- そこでデバイスを、他のネットワークドライバの隣、`drivers/net/wireguard` に移し、小さな `wg` コマンドが ioctl(入出力制御)で設定する形にしました。Version 1 は、プロトコルが NuttX の上で動くことを示しました。 Version 2 は、NuttX そのものに合わせることです: どのビルドモードでも使える普通のドライバで、コミュニティが レビューしやすい形にすること。受け入れられるという意味ではありません。NuttX の作りに沿う、という意味です。

26 — カーネル側の設計: 全体像 (19:13 → 20:04)

画面: 見出し「The Kernel Design: The Big Picture」。図は2つの帯 — 上が USER SPACE(青みがかった地、 `wg` コマンドだけ。apps/system/wg、genkey・set・show)、下が NUTTX KERNEL(緑がかった地)。 ioctl(ポインタを含まない平らな構造体1つ。秘密鍵は書き込み専用)がユーザー/カーネルの境界を越えて、カーネル内の `wg0`(drivers/net/wireguard)に届く。`wg0` は自分の UDP ソケットと受信スレッドを持ち、タイマー・ピア・ クッキーを管理し、NuttX の暗号(crypto/)を使う。左に NuttX のネットワークスタック、右に暗号化された UDP(wlan0 / usrsock → ピア)。

話す:

- 実際のドライバの形です。上がユーザー空間、下が NuttX カーネルです。
- ユーザー空間にあるのは、小さな `wg` コマンドだけです。これが ioctl でカーネルに設定を送ります。ioctl が運ぶのは、中にポインタを含まない平らな構造体1つなので、PROTECTED や KERNEL ビルドでも、カーネルがユーザーメモリへのポインタをたどる必要はありません。秘密鍵は書き込み専用です: どの ioctl も、それを返しません。
- カーネルの中では、`wg0` は普通のネットワークデバイスです。UDP ソケットと受信スレッドを自分で持っているので、Version 1 のスレッドとソケットのバグは、設計上もう起きません。暗号処理は NuttX 自身のものです。プロトコル本体は 今も wireguard-lwip 由来の BSD-3-Clause のコードで、そのことは LICENSE ファイルに書いてあります。

27 — Two Issues Found During Kernel Integration: カーネル統合で見つかった2つの問題 (20:04 → 21:32)

画面: 見出し「Two Issues Found During Kernel Integration」。左右2つの枠にラベル。左(青い縁、少し強調)CRYPTOGRAPHY — ChaCha20-Poly1305(NuttX `crypto/chachapoly.c`: ナンスのカウンタが RFC 8439 / WireGuard の並びと違うバイトにある。カウンタ0では一致するので、ハンドシェイクは通るが、最初より後のデータパケットが失敗する。テストベクタつきの2行の修正を、独立した変更に)/ 右(灰色の縁)KERNEL STACK — Stack Overflow(このドライバ、KERNEL ビルドのみ: 3KB のカーネルスタックに 6KB の配列 → あふれてパニック。sim では出ない。配列はヒープへ。ドライバ自身のバグ)。背景は両方ともニュートラル。SPRESENSE の RTC_HIRES は画面に載せず、ノートだけ。最下段:「Porting WireGuard also surfaced findings worth sharing with the NuttX community.」

話す:

- カーネルへ移す途中で、2つの別々の問題が見つかりました。
- 1つ目は、NuttX の ChaCha20-Poly1305 のコードです。64ビットのカウンタが、RFC 8439(WireGuard が使う並び)とは 違うナンスのバイトに入ります。カウンタが0のときは両方の並びが同じバイト列になるので、ハンドシェイクは通り、その後、最初より後のデータパケットが復号に失敗します。修正は2行で、テストベクタをつけた独立した crypto の変更に しました。私の見る限り、現在の master も同じ並びです。
- 2つ目は、私自身のドライバのバグで、NuttX のバグではありません。ピアの設定が、3キロバイトのカーネルスタックに 6キロバイトの配列を置いていて、パニックを起こしました。KERNEL ビルドでだけです。スタックが大きい sim では 出ませんでした。配列はヒープに移しました。バグ4と同じスタックの教訓が、一段深いところで起きたものです。
- もう1つ、ボード側から。画面には載せていません: NuttX 13.0.1 と master で、高分解能 RTC のオプション `CONFIG_RTC_HIRES` を有効にすると SPRESENSE が起動しませんでした。原因を見つけて手元で直しました。報告は 下書きまでで、まだ出していません。
- これらは、コミュニティがレビューしやすい小さな単位に分けて用意したものです。どれもまだ提出していません。

言い方に注意: スタックの件は NuttX ではなくこのドライバの問題。提出済みと言わない。

28 — 検証: どこで何が動いたか (21:32 → 23:17) *(核心・削らない)*

画面: 見出し「Testing: What Ran Where」。表(Build mode / Environment / Result): FLAT — ESP32-S3・SPRESENSE (実機) ✓ Verified / KERNEL — QEMU rv-virt(Linux へのトンネル、4 CPU で負荷) ✓ Verified / PROTECTED — QEMU rv-virt(1件の例外を見つけて修正、20/20) ✓ Verified / KERNEL・PROTECTED — 実機 — Not tested yet。小さな注記「✓ は、ここに書いた条件で通ったという意味で、他のハードウェアへの保証ではない」。GitHub への案内と QR コードは置かない。最下段は2行: 実機の FLAT と QEMU の KERNEL / PROTECTED で検証した / それらの実機での検証は残っている。

話す:

- どのビルドモードがどこで動いたかを、正確に言います。3つあって、混ぜません。
- FLAT、実機で。10月12日に、アップストリーム向けに用意した提出シリーズそのものを、スタックちゃんの中の ESP32-S3 で 動かしました: Linux の WireGuard とハンドシェイクし、両方向の ping を、パケットの最大サイズまで通しました。以前の ビルドは、両方のボードで本物の Wi-Fi 越しに Windows クライアントと動いています。SPRESENSE ではメモリを測りました: 40回の down/up の後でも、ぴったり同じでした。受信スレッドは 6KB の既定値のうち 1,472 バイトを使っていました。その既定値は以前は勘でした。今は測定値です。

- KERNEL、QEMU で。`wg` コマンドは別のプログラムで、システムコールでドライバに届き、トンネルは Linux まで届きます。4 CPU で、設定を変えながら大量に流しました。例外は出ませんでしたが、これは1回のスクリプト実行で、競合の全探索 ではありません。
- PROTECTED、これも QEMU で。トンネルは動き、すべての ioctl がメモリ境界を越えます。繰り返すと、たまに例外が出ました。原因は `wg` コマンドのスタックあふれで、このモードではシステムコールが呼び出し側のスタックで動くためです。既定値を 大きくして直りました: 20回中20回。
- 見せていないのは、KERNEL と PROTECTED の実機です。PROTECTED を CoreS3 で試しましたが、このボードは ブートローダ経由のイメージを一切起動できず、WireGuard の問題ではありません。だからチェックマークは、「この条件で通った」という意味です。すべてのボードへの約束ではありません。記録は GitHub にあります。

29 — 目次 ⑥Live Demo (23:17 → 23:27)

画面: 目次と同じリスト。⑥Live Demo だけハイライト、他は薄い。

話す(一言): ここまでは技術の話でした。第6部: 実際に動くところを見てもらいます。

間: ハイライトされた行を指して一拍。

30 — ライブデモ: Stack-chan を WireGuard 越しに (23:27 → 25:57)

画面: 見出し「Live Demo: Stack-chan over WireGuard」。1枚だけで、デモ中はこの1枚を出したまま、端末を横に並べる。上半分 NETWORK SETUP: Laptop(WireGuard peer、VPN IP 10.10.0.1)と Stack-chan(Apache NuttX + WireGuard、VPN IP 10.10.0.2)が、Smartphone(Wi-Fi hotspot、ローカルネットワークを提供)に点線の Wi-Fi でつながり、ラップトップと Stack-chan の間に太い赤の「Encrypted WireGuard Tunnel」が1本。スマホは WireGuard のピアとして描かない。下半分 LIVE DEMO STEPS: 1 VPN Connection(`wg show`)/ 2 Remote Login(`telnet 10.10.0.2`)/ 3 Network Interface(`ifconfig` → `wg0`)/ 4 Control Stack-chan(表情を変える・声を出す)。隅に小さな Stack-chan の写真(GitHub へのリンク)。Version の表記、Takeaway の枠、動画へのリンクは置かない。

話す:

- これからライブデモです。このスライドは出したままにして、端末をその横に置きます。
- まず構成です。ラップトップと Stack-chan は、どちらも私のスマホの Wi-Fi テザリングにつないでいます。スマホはローカル ネットワークを提供するだけです。ラップトップと Stack-chan の間には WireGuard のトンネルが1本あります: ラップトップが 10.10.0.1、Apache NuttX で動く Stack-chan が 10.10.0.2。スマホは WireGuard のピアではなく、暗号化された UDP を運ぶだけです。
- ステップ1、VPN の接続。ラップトップで `wg show` を打ちます。ピア、最新のハンドシェイク、スマホのネットワーク上のエンドポイント、転送量のカウンタが見えるはずです。
- ステップ2、リモートログイン。`telnet 10.10.0.2` — トンネルの中のアドレスです。NuttX のシェルのプロンプトが見えるはずです。
- ステップ3、ネットワークインターフェース。`ifconfig` を打ちます。スマホのネットワーク上の wlan0 と、10.10.0.2 を持つ wg0 が見えるはずです。つまり、本当にトンネル越しにログインしています。
- ステップ4、操作。`stackchan face happy` で表情が変わります。ほかの表情は neutral、sad、angry、sleepy、surprised です。次に `stackchan say` で、ラップトップにある音声ファイル http://10.10.0.1:8000/hello.wav を指定します。ファイルは同じトンネルを 通って届き、Stack-chan が話して口が動きます。音声は事前にラップトップで作ったもので、クラウドの API は使っていません。

- 時間があれば、パケットを見せます。別の端末で、スマホのネットワーク側のキャプチャは、ポート 51820 の暗号化された UDP だけ を示します。トンネルの中のキャプチャは、telnet の文字列が平文で見えます。そのあと、もう一度 `wg show` で、転送バイト数が 増えています。
- このロボットは、アプリケーション版の FLAT ビルドで動いています。カーネルドライバは試験の部で見たとおり同じ ESP32-S3 で動き ましたが、このロボットのデモはアプリケーション版の上に作っています。持ち帰ってほしいのは、NuttX の機器を WireGuard 越しに 遠隔操作できる、ということです。では、まとめです。

ライブ: `scripts/stackchan/demo.sh` を使う。手順1は `show`、手順2は `login`(そのあと telnet の中で `ifconfig`、`stackchan face happy`、`stackchan say http://10.10.0.1:8000/hello.wav`)、任意で `outside` と `inside` を別の端末2つで。画面の半分に端末、もう半分にこのスライド。

本番前: スマホのテザリングを ON(2.4 GHz / 互換性優先)、ラップトップを接続、Docker Desktop を起動。Stack-chan を `nsh_wifi.py` でつなぐ(パスフレーズはボードに保存しない)、`wg_setup.py` を1回実行。`stackchan say` を1回試す。テザリングを 入れ直すとアドレス帯が変わることがあるので、そのときは `nsh_wifi.py` と `wg_setup.py` をやり直す。`say` の途中でボードが 止まることがまれにある(約20回に1回)。止まったら電源を入れ直して同じ手順。

画面に出さない: `wg showconf`(秘密鍵を表示する)、`%USERPROFILE%\stackchan-wg` の鍵ファイル、テザリングのパスフレーズ。

ファームウェア: このロボットはアプリケーション版(FLAT)。Dockerfile の `esp32s3-stackchan` ステージで作る。カーネルドライバは 別のビルド(2026-10-12)で同じ ESP32-S3 で動かしたが、Stack-chan のアプリは入っていない。

フォールバック: 今のところバックアップの動画はない。ライブが止まったら、そう言ってまとめへ進む。

31 — 目次 ⑦Summary & Reflections (25:57 → 26:07)

画面: 目次と同じリスト。⑦Summary & Reflections だけハイライト、他は薄い。

話す(一言): 最後の部: まとめ、学んだこと、そして感謝。

間: ハイライトされた行を指して一拍。

32 — Summary (26:07 → 26:41)

画面: 見出し「Summary」。3つの行で、それぞれ短い見出しと1文: ① Secure Communication(WireGuard を Apache NuttX に持ち込み、標準の WireGuard ピアと安全な IP 通信ができるようにした)/ ② Application to Kernel Driver(アプリケーションからカーネルの ネットワークドライバへ発展させ、実機と QEMU で検証した)/ ③ Toward Upstream Contribution(ドライバと関連する修正を、Apache NuttX コミュニティのレビューに向けて準備している。小さく「小さな別々の変更として用意した。まだ何も提出していない」)。`wg0` などの内部用語や、PR の分け方・ファイルパスは載せない。

話す:

- まとめます。私たちは WireGuard を Apache NuttX に持ち込みました。NuttX の機器が、標準の WireGuard ピアと、暗号化された IP トンネルで話せるようになりました。
- アプリケーションからカーネルのネットワークドライバへ発展させました。実機で、そして QEMU のカーネル系のビルドモードで 検証しました。

- 次に、ドライバと関連する修正を、NuttX コミュニティのレビューに向けて準備しています。小さな別々の単位で、まだ何も 提出していません。ここまでが技術の話です。次は、私が学んだことです。

「マージされたか」と聞かれたら: まだ。何も提出していない。提出の候補であって、マージできるという主張ではない。Q&A 資料を参照。

提出の状況(2026-10-13): apache/nuttx にも apache/nuttx-apps にも、この作業の pull request は無い。関連する issue は閉じた GSoC の提案スレッド(#18548)だけ。ブランチは発表者のフォークにあるだけ。「提出した」と言わない。

33 — What I Learned: 学んだこと (26:41 → 28:01)

画面: 見出し「What I learned」。カード3枚: 1 アプリのユーザーから OS のコントリビューターへ / 2 AI はコードを書くのを助けてくれたが、理解するのは私の仕事だった(強調)/ 3 歓迎してくれるオープンソースコミュニティ(良いドキュメントと貢献ガイド、活発な議論とレビュー、人が責任を持つ AI 支援開発への前向きさ、CI・ハッカソン・イベントが新しい人の参加を助ける)。最下段:「I hope to keep contributing to Apache NuttX and the open-source community, and I look forward to seeing NuttX continue to grow.」

話す:

- ここまでが技術の話でした。次は、学んだことを3つ。
- 1つ目。私は NuttX にアプリケーション開発者として来ました。今回は OS の中で、ドライバとカーネルを扱いました。
- 2つ目。これははっきり言いたいです。このプロジェクトでは AI、Claude をたくさん使いました。調査もコードも、ずっと速くなりました。でも組み込み OS の内部は私には新しく、設計と動きを本当に理解するのは大変でした。AI が書いたコードをそのまま 受け取ることはできませんでした。質問し、NuttX のソースを読み、動きを試し、自分で説明できるようになるまで続けました。AI はコードを書くのを助けてくれましたが、理解するのは私の仕事でした。
- 3つ目、コミュニティです。NuttX には良いドキュメントと貢献ガイドがあり、技術的な議論とレビューが活発です。AI 支援の 開発にも前向きですが、責任は人が持ちます: コミットには Assisted-by を書き、AI は Signed-off-by を付けません。そして CI や ハッカソンが、新しい人の参加を助けています。オープンソースは、コードが公開されているだけのものではありません。新しい人が 加わり、学び、貢献できるときに育ちます。
- NuttX と、オープンソースコミュニティに貢献し続けたいと思っていますし、NuttX がこれからも育っていくのを楽しみにしています。

AI の話は率直に。 告白でも自慢でもなく: 速くなった、でも理解は難しく、やはり自分の仕事だった。

34 — Acknowledgements: 謝辞 (28:01 → 28:40)

画面: 見出し「Acknowledgements」。カードをやめ、縦のシンプルな一覧。各項目は、名前(太字、少し大きく)と短い1行: Alan Carvalho de Assis(この発表を Community Over Code に出すよう勧めてくれた)/ The Apache NuttX Community (議論、ドキュメント、支援)/ Sony's NuttX Developers, Past and Present(NuttX への貢献と、築いてくれた土台)/ The Apache Software Foundation & Community Over Code Organizers(オープンソースのコミュニティをつなげてくれている。運営者もボランティアも)。順番は 個人 → コミュニティ → 関係する組織。

話す:

- 最後に、感謝を伝えます。

- まず、Alan Carvalho de Assis さん。この発表を Community Over Code に出すよう勧めてくれました。次に、Apache NuttX コミュニティ。議論、ドキュメント、支援をありがとうございました。GSoC の提案を書いたとき、皆さんが話を聞いて励ましてくれて、 選ばれなかった後も続けられたのは、そのおかげです。
- 3つ目に、私より前の Sony のエンジニアたち。NuttX に貢献し、この仕事が立つ土台を築いてくれました。そして最後に、Apache Software Foundation と、Community Over Code の運営者とボランティアの皆さん。オープンソースのコミュニティをつなげてくれて ありがとうございます。

35 — Thank You — Questions? (28:40 → 28:57)

画面: 見出し「Thank You — Questions?」。大きな QR コード(「Scan to view the slides」)と、その横にクリックできるリンク2つ: スライド(https://wwlapaki310.github.io/gsoc2026-nuttx-wireguard/docs/presentation/coc-glasgow-reveal/index.html)と GitHub のリポジトリ。表紙にも同じ QR コード(「Slides」)。動画のリンクは置かない。

話す:

- ありがとうございました。カーネルドライバ、バグ、ハードウェアについて、質問をお待ちしています。
- スライドは画面にあります。QR コードを読み取るか、リンクを使ってください。コードと記録は GitHub にあります。

終わった後: coc-glasgow-qa.md を手元に。よく出る質問(リベースの状況、#14、IOB の枯渇、checkpatch)は そこに答えがあり、スライドには無い。

本番前に確認: QR コードと2つのリンクが、上の URL を開くこと(GitHub Pages が最後の push に追いついていること)。

A1 — 付録: 検証: ping が見逃すもの (付録・時間に含めない)

付録: 本編には含めない。Thank You の後、質問で必要になったら、最後のスライドから下矢印で開く。

画面: 見出し「Three problems that ping would never notice.」。カード3枚 — 起動のたびに同じ鍵? (SPRESENSE: 3回のリセットで3つの違う鍵。簡単な確認で、乱数の質の証明ではない)/ down の後に鍵が メモリに残る?(sim: セッション鍵は down の後すべてゼロ。静的鍵は up をやり直すために意図的に残す)/ 数時間後に出るリーク?(sim: 151回の再鍵化、100回の down/up、何も失われない。メモリは起動時の数に戻る。 1週間の試験ではない)。

話す:

- このスライドは、動くトンネルが教えてくれないことです: ping では決して気づかない3つの問題。
- まず鍵です。NuttX は `/dev/urandom` の seed をビルド時の設定から取ることができます。そうするとボードは起動のたびに同じ秘密鍵を作り、それでもすべて動きます。そこで SPRESENSE を3回リセットし、3つの違う鍵を得ました。簡単な確認であって、乱数の質の 証明ではありません。
- 2つ目、シミュレータでデバイスのメモリを読みました。down の後、セッション鍵は すべてゼロでした。静的鍵は、もう一度 up できるように、意図的に残しています。
- 3つ目、長い試験で、これもシミュレータです: 151回の強制的な再鍵化と、100回の down/up。バッファは1つも失われず、メモリは起動時とちょうど同じ数に戻りました。 1週間動いたとか、シミュレータがボードと同じように振る舞う、とは言っていません。

A2 — 付録: 設計判断: タイムスタンプ問題には楽な答えがない (付録・時間に含めない)

付録: 本編には含めない。Thank You の後、質問で必要になったら、最後のスライドから下矢印で開く。

画面: 規則(再起動の後でも、相手が見たものより新しいタイムスタンプが要る)。Chosen(リアルタイム クロック + この起動で使った最大値。Linux・wireguard-go と同じ規則 + 時計が未設定なら警告)/ Not chosen (毎回保存: flash が摩耗 / flash に範囲を予約: 確かめられなかった書き込み保証が要る / 待つ: 前回動いて いた時間だけ)。実測のペア(時計未設定 → 75秒でハンドシェイクなし、時計設定 → 4.1秒)。

話す:

- ここは試験ではなく、判断が必要だった唯一の場所です。そして、その判断を測りました。
- WireGuard のリプレイ防御には規則があります: すべてのハンドシェイクは、相手が これまでに見たものより新しいタイムスタンプを持たなければならない — 再起動の 後でもです。デバイスは普通ハンドシェイクを始める側で、しかもバッテリー付きの 時計を持っていない可能性が最も高い側です。
- 毎回のタイムスタンプを保存すると flash が摩耗します。flash に範囲を予約するのが 長期的には正しい答えですが、それには、これらのファイルシステムで確かめられなかった 書き込みの保証が必要です。そしてただ待つなら、1か月動いていたデバイスは、1か月 再接続できません。ここでは、楽な選択と正しい選択が一致しませんでした *(ダンブルドアの「正しいことと楽なこと」)*。
- そこで、リアルタイムクロックと、この起動の中で使った最大値を選びました — Linux と wireguard-go とまったく同じです。そして時計が一度も設定されていなければ、ドライバが 一度だけ警告を出します。
- それから実機で測りました。ボードには何も送りませんでした。相手からの ping は、ボードの代わりにハンドシェイクを始めてしまうからです。起動時のままの時計: 75秒 見ていてもハンドシェイクなし。時計を設定すると: 4.1秒。だからドライバは警告を出します。